

- Description

This module manages a smart water pump monitoring system, ensuring that:

1. **The water flow rate is continuously measured and compared against efficiency thresholds.**
2. **If the flow rate is too low, a clogging or low-pressure warning is triggered.**
3. **If the energy consumption exceeds a set threshold, an optimization alert is issued.**
4. **A log of all flow measurements, energy usage, and efficiency recommendations is maintained for analysis.**

This system helps optimize water pump performance, reduce energy waste, and ensure stable water distribution.

- Specification

module SmartWaterPumpMonitoring

var currentFlowRate, minFlowRate, currentEnergyUsage, maxEnergyUsage, runTime:
real

var log: sequence of string

process updateFlowRate(changeAmount: real)

functional scenario form:

currentFlowRate $\geq$ minFlowRate && log = log $\oplus$ ("Flow Rate Updated: " + currentFlowRate + " L/min"), Flow rate updated successfully.

||

currentFlowRate < minFlowRate && log = log $\oplus$ ("Low Flow Alert: " + currentFlowRate + " L/min, Possible Clogging or Low Pressure."), Low flow alert triggered, possible clogging.

end process

process calculateEnergyUsage(runTime: real)

functional scenario form:

(currentEnergyUsage $\leq$ maxEnergyUsage) && log = log $\oplus$ ("Energy Consumption Recorded: " + currentEnergyUsage + " kWh"), Energy usage recorded successfully.

OR

(currentEnergyUsage > maxEnergyUsage) && log = log $\oplus$ ("High Energy Usage Alert: " + currentEnergyUsage + " kWh, Efficiency Optimization Required."), High energy usage alert triggered.

end process

end module

- Java Code

```
package code;
import java.util.*;
class WaterPump {
```

```

        private double minFlowRate;
        private double maxEnergyUsage;
        private double currentFlowRate;
        private double currentEnergyUsage;
        private List<String> log;
        public WaterPump(double minFlowRate, double maxEnergyUsage, double
initialFlowRate) {
            this.minFlowRate = minFlowRate;
            this.maxEnergyUsage = maxEnergyUsage;
            this.currentFlowRate = initialFlowRate;
            this.currentEnergyUsage = 0.0;
            this.log = new ArrayList<>();
        }
        public void updateFlowRate(double changeAmount) {
            currentFlowRate += changeAmount;
            String message = String.format("Current Flow Rate: %.2f L/min",
currentFlowRate);
            if (currentFlowRate < minFlowRate) {
                message += " | ALERT: Low Flow Detected! Possible Clogging
or Low Pressure.";
            }
            System.out.println(message);
            log.add(message);
        }
        public void calculateEnergyUsage(double runTime) {
            currentEnergyUsage = (currentFlowRate / minFlowRate) * runTime
* 1.5;
            String message = String.format("Estimated Energy Usage: %.2f kW
h for %.1f minutes operation", currentEnergyUsage, runTime);
            if (currentEnergyUsage > maxEnergyUsage) {
                message += " | WARNING: High Energy Consumption! Consider E
fficiency Optimization.";
            }
            System.out.println(message);
            log.add(message);
        }
        public void printLog() {
            System.out.println("\nWater Pump System Log:");
            for (String logEntry : log) {
                System.out.println(logEntry);
            }
        }
    }
}
public class c36_SmartWaterPumpSystem {

```

```

    public static void main(String[] args) {
        WaterPump pump = new WaterPump(30.0, 10.0, 50.0);
        // testcase-VT:
        //     pump.updateFlowRate(-10.0);
        //     pump.updateFlowRate(-15.0);
        //     pump.calculateEnergyUsage(20.0);
        //     pump.calculateEnergyUsage(60.0);
        //     pump.calculateEnergyUsage(1.0);
        //     pump.printLog();

        // testcase-FT:
        //     pump.updateFlowRate(-1.7976931348623157E308);
        //     pump.updateFlowRate(0.0);
        //     pump.updateFlowRate(2.4850550729069995E291);
        //     pump.updateFlowRate(6.384642474568709E293);
        //     pump.updateFlowRate(7.016626088207998E290);
        //     pump.printLog();

        // testcase-Z3:
        pump.updateFlowRate(-100);
        pump.updateFlowRate(-15);
        pump.calculateEnergyUsage(20);
        pump.calculateEnergyUsage(600);
        pump.calculateEnergyUsage(1);

        // testcase-UVT:
        //     pump.updateFlowRate(10);
        //     pump.updateFlowRate(15);
        //     pump.calculateEnergyUsage(20);
        //     pump.calculateEnergyUsage(60);
        //     pump.calculateEnergyUsage(1);
    }
}

```

● Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	838	844
▼ src	0.7 %	6	838	844
▼ code	0.7 %	6	838	844
▼ c36_SmartWaterPumpSystem.java	100.0 %	6	0	6
> c36_SmartWaterPumpSystem		0	0	0
> WaterPump	100.0 %	6	0	6

3. Vibration testing instruction coverage: 98.0%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	144	20,218	20,362
✓ src	0.7 %	144	20,218	20,362
✓ code	0.7 %	144	20,218	20,362
✓ c36_SmartWaterPumpSystem.java	98.0 %	144	3	147
> c36_SmartWaterPumpSystem	89.3 %	25	3	28
> WaterPump	100.0 %	119	0	119

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.4 %	3	841	844
✓ src	0.4 %	3	841	844
✓ code	0.4 %	3	841	844
✓ c36_SmartWaterPumpSystem.java	50.0 %	3	3	6
> c36_SmartWaterPumpSystem		0	0	0
> WaterPump	50.0 %	3	3	6

5. Fuzz testing instruction coverage: 67.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	99	20,263	20,362
✓ src	0.5 %	99	20,263	20,362
✓ code	0.5 %	99	20,263	20,362
✓ c36_SmartWaterPumpSystem.java	67.3 %	99	48	147
> c36_SmartWaterPumpSystem	89.3 %	25	3	28
> WaterPump	62.2 %	74	45	119

6. Z3 branch coverage: 33.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.2 %	2	838	840
✓ src	0.2 %	2	838	840
✓ code	0.2 %	2	838	840
✓ c36_SmartWaterPumpSystem.java	33.3 %	2	4	6
> WaterPump	33.3 %	2	4	6
> c36_SmartWaterPumpSystem		0	0	0

7. Z3 instruction coverage: 82.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	120	22,266	22,386
✓ src	0.5 %	120	22,266	22,386
✓ code	0.5 %	120	22,266	22,386
✓ c36_SmartWaterPumpSystem.java	82.8 %	120	25	145
> WaterPump	81.5 %	97	22	119
> c36_SmartWaterPumpSystem	88.5 %	23	3	26

8. Unimproved V-method branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.4 %	3	837	840
✓ src	0.4 %	3	837	840
✓ code	0.4 %	3	837	840
✓ c36_SmartWaterPumpSystem.java	50.0 %	3	3	6
> WaterPump	50.0 %	3	3	6
> c36_SmartWaterPumpSystem		0	0	0

9. Unimproved V-method instruction coverage: 82.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	120	22,266	22,386
✓ src	0.5 %	120	22,266	22,386
✓ code	0.5 %	120	22,266	22,386
✓ c36_SmartWaterPumpSystem.java	82.8 %	120	25	145
> WaterPump	81.5 %	97	22	119
> c36_SmartWaterPumpSystem	88.5 %	23	3	26